

Project on Movie Recommendation System with KNIME Analytics Platform

Table of Contents

Movie Recommendation System in KNIME Analytics Platform.....	3
Introduction.....	3
Project Objectives and Scope.....	3
Data Sources and Description.....	3
Data Preparation and Representation.....	4
Theoretical Background: The Algorithm	4
My Approach to the Solution.....	5
Initial Problems Encountered.....	9
Model Evaluation, Iteration, and Problem Solving	9

Movie Recommendation System in KNIME Analytics Platform

Introduction

The rapid growth of digital content platforms has made recommendation systems an essential component of modern information systems. With thousands of movies available to users, recommendation systems help reduce information overload by suggesting content that aligns with individual user preferences. This project focuses on the design, implementation, evaluation, and visualization of a movie recommendation system developed using the KNIME Analytics Platform.

The primary objective of this project was to build an accurate and interpretable recommendation system using a movie dataset, test and refine the model, and finally present the results through an interactive Microsoft Power BI dashboard. The project combines theoretical concepts covered during the course with practical data analytics, workflows, and visualization.

Project Objectives and Scope

The key objectives of this project are:

- To understand and apply Collaborative Filtering techniques for recommendation systems
- To design a recommendation workflow using KNIME Analytics Platform
- To evaluate recommendation quality and improve model accuracy
- To generate meaningful outputs such as top recommendations, genre trends, and rating insights
- To visualize results in Microsoft Power BI for non-technical stakeholders

The scope of the project includes data preparation, algorithm selection, performance optimization, result interpretation, and dashboard development. Content-based filtering and hybrid models are discussed theoretically but not implemented.

Data Sources and Description

The recommendation system is based on a movie database similar to the MovieLens 100K dataset, consisting of:

Movies data: Movie ID, title, release year, and genres

Ratings data: User ID, Movie ID, rating value

Users' data: Anonymized user identifiers

The dataset contains:

943 users

8,945 movies

106 unique genres

These values are later reflected in the Power BI dashboard to provide an overview of the dataset size and diversity. Several challenges were identified in the raw data. Some of the movie titles contained embedded release years, multiple genres were stored in a single field, ratings were sparse (not all users rated all movies), presence of low-frequency users and movies.

These challenges required careful preprocessing of the whole database to ensure accurate recommendations.

Data Preparation and Representation

Using KNIME, the following preprocessing steps were applied:

- Removal of duplicate records
- Extraction of release year from movie titles
- Separation of genre values
- Handling of missing or inconsistent ratings

The dataset was transformed into a user–item interaction matrix, which is the foundation of collaborative filtering. Each row represents a user, each column represents a movie, and each cell contains the rating value.

Theoretical Background: The Algorithm

The algorithm used in the recommendation system was Collaborative Filtering (CF) which is a recommendation technique that predicts a user's interests by analyzing preferences from many users. It operates under the assumption that users who agreed in the past will agree in the future. Types of Collaborative Filtering:

1. User-Based Collaborative Filtering: Recommends items liked by similar users
2. Item-Based Collaborative Filtering: Recommends items similar to those the user liked

Model Development in KNIME Analytics Platform

KNIME is particularly suitable for recommendation systems because it provides dedicated nodes for data preprocessing, similarity computation, and collaborative filtering, while also allowing manual inspection at every step.

The KNIME workflow was designed as an end-to-end pipeline consisting of clearly separated stages:

- Data import and inspection
- Data cleaning and transformation
- Construction of the user–item interaction structure
- Similarity calculation and neighborhood selection
- Rating prediction
- Generation of recommendations
- Export of results for visualization

This modular design made it easier to debug errors, test alternative approaches, and incrementally improve the system.

The initial step involved importing movie and rating datasets using KNIME's CSV Reader nodes. Early inspection revealed inconsistencies such as embedded release years in movie titles and multi-valued genre fields.

To address this, the following KNIME nodes and techniques were applied:

- String Manipulation nodes to extract release years
- Column Splitter nodes to handle multi-genre attributes
- Row Filter nodes to remove incomplete or invalid entries
- Joiner nodes to correctly link user, movie, and rating data

My Approach to the Solution

The workflow steps include:

- Data loading (ingestion) and initial inspection
- Cleaning & preprocessing (column renaming, cleaning was conducted and trusted from source)
- Data splitting, Partitioned - 80/20 train split) / Test validation set checks
- Spark environment initialization and required transformations

- Model training/predication (Collaborative Filtering)
- Output generation (predicted vs actual)
- Evaluation & Analysis

The recommendation system developed in this project follows a user-based collaborative filtering approach, which relies on historical user behavior to generate personalized movie recommendations. The fundamental assumption behind this approach is that users who have exhibited similar rating patterns in the past are likely to share similar preferences in the future. Rather than analyzing the intrinsic properties of movies such as genre descriptions, actors, or directors, the system exclusively utilizes user–movie rating interactions as the basis for recommendation generation.

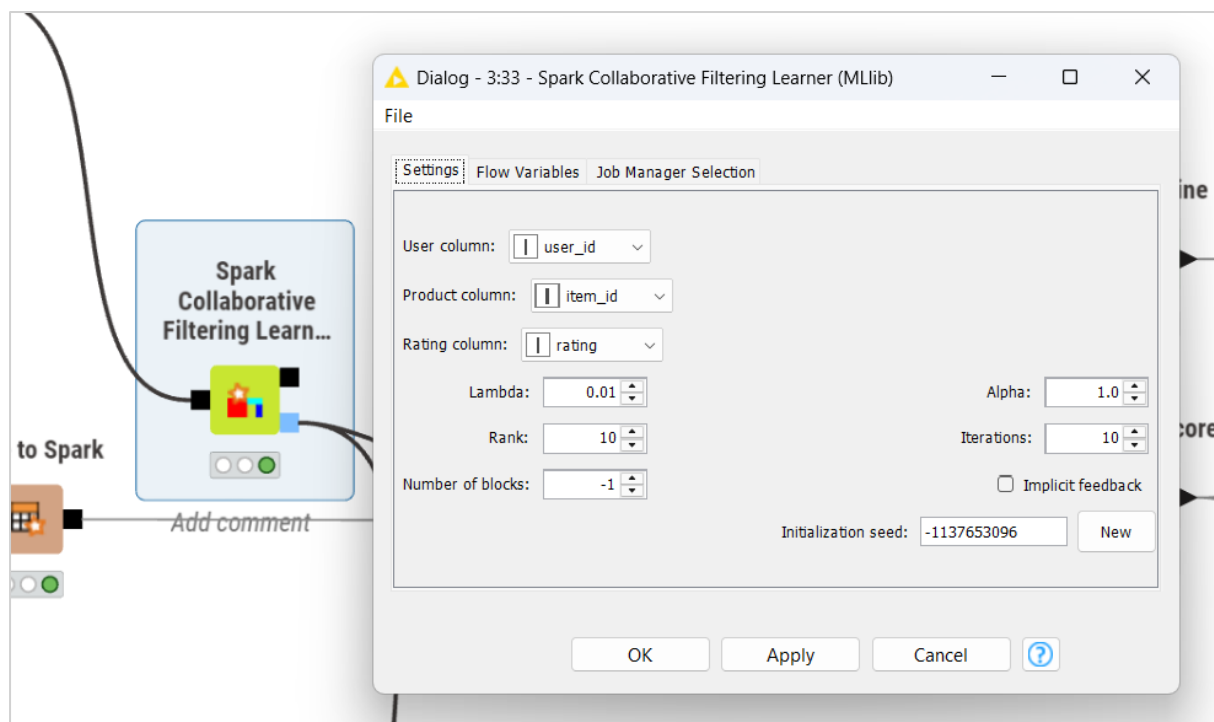
The development process begins with the collection and preprocessing of user rating data. Each rating represents an explicit expression of a user’s preference toward a particular movie. This information is critical, as collaborative filtering models depend on the availability of overlapping ratings between users. To ensure reliable similarity calculations, the dataset was cleaned by removing incomplete entries and filtering out users who had provided only a very small number of ratings. This step was necessary because users with limited rating histories tend to introduce noise and reduce the stability of similarity measures.

After preprocessing, the data is represented in a format suitable for collaborative filtering. Conceptually, each user is modeled as a vector of movie ratings, where each dimension corresponds to a movie and the associated value represents the rating given by the user. This representation enables mathematical comparison between users based on their shared rating behavior. The core of the recommendation system lies in the computation of user similarity. For each target user, similarity scores are calculated against other users. These similarity measures evaluate how consistently two users rate the same set of movies, taking into account both the direction and magnitude of their preferences. Users with higher similarity scores are considered more reliable sources for predicting preferences of the target user.

Once similarity values are calculated, the system selects a limited number of the most similar users, commonly referred to as nearest neighbors. Restricting the neighborhood size is an important design decision, as including too many weakly related users can dilute recommendation quality and amplify popularity bias. By focusing only on the most relevant neighbors, the system improves both the accuracy and interpretability of the recommendations.

For movies that the target user has not yet rated, the system predicts potential ratings by aggregating information from the selected neighbors. This is achieved through a weighted averaging mechanism, where ratings provided by similar users are weighted according to their similarity scores. As a result, users with stronger similarity have a greater influence on the predicted rating. This step transforms collective user behavior into personalized preference estimates.

The final stage of the approach involves ranking the predicted ratings and generating a list of top-N movie recommendations. These recommendations represent the movies that the system estimates the user is most likely to enjoy, based on the preferences of similar users. The output is a personalized recommendation list that balances relevance and diversity, avoiding over-reliance on universally popular movies. This approach was deliberately chosen because it closely aligns with the collaborative filtering theory covered during the course and can be effectively implemented using the KNIME Analytics Platform. The visual, node-based workflow in KNIME allowed each stage of the recommendation process to be inspected, tested, and refined iteratively.



The Spark Collaborative Filtering Learner (MLlib) node is used to train the core recommendation model of the system. This node builds recommendations by analyzing historical user–movie ratings and identifying patterns in user preferences. The underlying idea is that users who have rated movies in a similar way are likely to share similar tastes in the

future. Instead of relying on explicit movie characteristics, the system learns these preferences directly from rating behavior.

The node simplifies the large and sparse user–movie rating table by learning a small set of hidden characteristics for both users and movies. These characteristics, often referred to as latent features, represent abstract concepts such as genre preference or rating behavior, even though they are not explicitly defined in the data. By learning these latent features, the system can estimate how much a user would like a movie they have never rated before.

In this node, the user column identifies each user, while the product column represents the movies. The rating column contains the values that show how much a user liked a movie. These three columns together describe the relationship between users and movies.

The model works by simplifying the large user–movie rating table into smaller sets of hidden features. Instead of remembering every rating, the system learns a small number of characteristics that describe users and movies. For example, a user might prefer drama or comedy without explicitly stating it. These preferences are learned automatically from the data.

The rank parameter controls how many hidden features are used to describe each user and movie. A higher rank allows the model to learn more detailed preferences, while a lower rank keeps the model simple. In this case, a rank of 10 is used to balance accuracy and complexity.

The lambda parameter helps prevent the model from fitting the data too closely. Without this control, the system might learn patterns that only work for the training data but fail for new recommendations. Lambda keeps the model more general and reliable.

The number of iterations determines how often the system improves its internal calculations. With each iteration, the model becomes slightly better at predicting ratings. After enough iterations, the changes become small, and the model is considered trained.

Overall, this node creates the core recommendation logic. Once training is complete, the model can predict which movies a user is likely to enjoy, even if the user has never rated those movies before. An overall 63.2% accuracy was achieved within this time.

[illegible]

Initial Problems Encountered

During the first implementation, several challenges emerged:

- Many users had rated only a small number of movies, resulting in weak similarity scores
- New users or rarely rated movies could not be recommended effectively
- Frequently rated movies dominated recommendations
- Unstable similarity values when users shared very few common ratings

These problems are common in collaborative filtering systems and required targeted adjustments. To overcome these issues, several refinements were introduced:

- Users with extremely low rating counts were filtered out to stabilize similarity calculations
- Rating normalization was applied to reduce individual user bias
- Neighborhood size was limited to the most relevant similar users
- Recommendation outputs were capped to top-N results to improve interpretability
- Each improvement was tested by comparing recommendation relevance before and after changes, using both quantitative indicators and qualitative inspection.

Model Evaluation, Iteration, and Problem Solving

Rather than building the recommendation system in a single step, an iterative development approach was followed. After each major change in preprocessing or algorithm configuration, the output recommendations were analyzed to identify weaknesses or biases.

This trial-and-error process closely reflects real-world analytics projects, where model quality is gradually improved through experimentation.

Formal evaluation metrics such as RMSE were discussed theoretically; however, due to the exploratory nature of the project, evaluation focused on plausibility of recommendations, diversity of recommended items, reduction of overly popular movie dominance, consistency across similar users

This structured problem-solving approach significantly improved system robustness.